МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления проектами и отслеживания задач»

Выполнил: Набиуллин Артур Фанилевич
Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Набиуллину Артуру Фанилевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления проектами и отслеживания задач»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	05.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	11.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования)	7	02.05.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	09.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Реализация подсистем аутентификации и авторизации пользователей	8	21.05.2026	
Задача 3.2. Реализация системы распределения ролей и проверки прав доступа к функциям приложения	7	26.05.2026	
Задача 3.3. Реализация CRUD-интерфейсов для управления проектами и задачами	7	30.05.2026	
Задача 3.4. Разработка функциональности загрузки и скачивания файлов, прикрепленных к задачам	12	03.06.2026	
Задача 3.5. Реализация модуля агрегирования данных для расчёта и вывода статистики по проектам	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	09.06.2026	
Задача 4.2. Деплой приложения на хостинг	4	11.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания « 31 » марта 2026 г.

Дата сдачи выполненной работы « 13 » июня 2026 г.

Задание принял к исполнению

« 31 » марта 2026 г. _____ А. Ф. Набиуллин
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **6**

1.1 Обзор существующих программных продуктов по теме работы 6

1.2 Анализ программных инструментов разработки веб-приложений 6

1.3 Формулировка цели и задач работы 8

2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ **10**

2.1 Анализ целевой аудитории 10

2.2 Описание функциональности приложения 11

2.3 Проектирование модели данных 13

2.4 Разработка макетов страниц 15

3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ **17**

3.1 Реализация подсистемы аутентификации и проверки ролей . . 17

3.2 Разработка CRUD-интерфейсов и работы с файлами 19

3.3 Реализация модуля агрегирования данных 20

ЗАКЛЮЧЕНИЕ **22**

ВВЕДЕНИЕ

Актуальность темы. В современных условиях цифровизации бизнеса и повсеместного перехода на удаленный или гибридный формат работы, эффективное управление проектами становится критически важным фактором успеха любой организации. Использование традиционных методов учета задач (электронные таблицы, мессенджеры) приводит к потере информации, срыву сроков и снижению производительности команды. Разработка специализированных веб-приложений для трекинга задач позволяет централизовать рабочие процессы, автоматизировать контроль поручений и обеспечить прозрачность работы на всех этапах жизненного цикла проекта.

Степень разработанности проблемы. На рынке существует множество систем управления проектами (Jira, Trello, Asana и др.). Однако большинство корпоративных решений обладают избыточным функционалом, сложны в освоении и имеют высокую стоимость лицензий, что делает их нерентабельными для малого бизнеса, студенческих команд или индивидуального использования. В связи с этим, создание легковесного, интуитивно понятного веб-приложения с базовым, но достаточным набором функций (CRUD-операции, ролевая модель, прикрепление файлов) остается актуальной практической задачей.

Объект исследования — процессы управления проектами, распределения задач и контроля их выполнения в рабочих группах.

Предмет исследования — программные средства, алгоритмы и методы веб-разработки, применяемые для создания систем отслеживания задач.

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения, предназначенного для эффективного управления проектами и отслеживания статусов выполнения задач с учетом ролевой модели доступа.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке программного обеспечения для управления проектами представлено множество решений. Для выявления недостатков существующих систем и формирования требований к разрабатываемому веб-приложению проведен структурированный анализ популярных платформ: Jira, Trello и Asana.

Сравнение проводилось по ключевым критериям, критически важным для небольших рабочих групп: наличие полнофункциональной бесплатной версии, простота освоения, поддержка ролевой модели доступа (RBAC), наличие встроенной статистики и возможность прикрепления файлов. Результаты сравнительного анализа представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ систем управления проектами

Критерий оценки	**Jira**	**Trello**	**Asana**	**Наш проект**
Бесплатный полный функционал	–	–	–	+
Простота освоения	–	+	+	+
Ролевая модель доступа	+	– (платно)	– (платно)	+
Графическая статистика	+	–	+	+
Прикрепление файлов	+	+	+	+

Анализ показал, что мощные корпоративные системы (Jira) обладают необходимым функционалом, но перегружены и сложны для малого бизнеса. Более простые решения (Trello, Asana) искусственно ограничивают функционал в бесплатных версиях, вынуждая покупать подписку ради доступа к статистике или ролевым моделям. Это подтверждает целесообразность разработки собственного легковесного веб-приложения.

1.2 Анализ программных инструментов разработки веб-приложений

Для реализации клиент-серверной архитектуры проекта необходимо выбрать оптимальный стек технологий. Сравнение производилось по трем ключевым направлениям: серверная часть (Backend), база данных и клиентская

часть (Frontend).

1. Выбор Backend-фреймворка. Рассматривались популярные решения на языке Python: *Django*, *FastAPI* и *Flask*.

- *Django* отличается монолитной архитектурой «всё включено», имеет встроенную ORM и панель администратора. Однако его избыточность негативно сказывается на скорости разработки небольших проектов.

- *FastAPI* выделяется асинхронной архитектурой и высокой производительностью, идеально подходит для построения микросервисов (API). Недостаток — сложность интеграции с классическим серверным рендерингом страниц.

- *Flask* [1] предоставляет минималистичное ядро с возможностью точечного подключения нужных библиотек (высокая модульность).

Вывод: Фреймворк Flask выбран как оптимальный баланс между легковесностью и функциональностью, позволяющий реализовать приложение без избыточного кода.

2. Выбор системы управления базами данных (СУБД). Сравнение проводилось между *SQLite*, *MySQL* и *PostgreSQL*.

- *SQLite* хранит данные в одном локальном файле, не требует настройки сервера, но блокирует базу целиком при записи, что исключает её использование в многопользовательской среде.

- *MySQL* обладает высокой скоростью чтения, однако исторически имеет менее строгую проверку типов данных и сложную систему изменения таблиц.

- *PostgreSQL* является мощной объектно-реляционной СУБД, поддерживающей механизм многоверсионности (MVCC) для параллельной работы без блокировок, строгую ссылочную целостность и сложные аналитические запросы.

Вывод: Для надежного хранения данных пользователей и работы с агрегацией статистики в production-среде выбрана СУБД PostgreSQL, взаимодействующая с кодом через ORM-библиотеку SQLAlchemy [2].

3. Выбор Frontend-технологий. Сравнивались подходы Single Page Application (SPA) и Server-Side Rendering (SSR).

- Использование SPA-фреймворков (*React* или *Vue.js*) позволяет создать высокоинтерактивный интерфейс без перезагрузки страниц, но требует настройки отдельного сервера сборки (Node.js/Webpack) и усложняет маршрутизацию запросов (CORS).
- Подход SSR на базе шаблонизатора *Jinja2* позволяет генерировать HTML-страницы непосредственно на Python-сервере. В связке с CSS-библиотекой *Bootstrap 5* это обеспечивает быструю отрисовку и адаптивность под мобильные устройства «из коробки».

Вывод: Для интерфейса Task Tracker'а динамики связки Jinja2 + Bootstrap 5 более чем достаточно, что исключает необходимость в избыточной SPA-архитектуре.

1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области (п. 1.1) и выбора технологического стека (п. 1.2), главная **цель работы** заключается в создании веб-приложения для управления проектами и отслеживания задач на базе фреймворка Flask и СУБД PostgreSQL.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и существующих решений;
2. Спроектировать пользовательские сценарии (Use Cases) и проанализировать целевую аудиторию;
3. Спроектировать логическую и физическую модели реляционной базы данных;
4. Разработать макеты пользовательского интерфейса (Wireframes);
5. Реализовать подсистему аутентификации пользователей и настроить систему распределения ролей;

6. Разработать программные модули (CRUD-интерфейсы) для управления сущностями «Проект», «Задача» и «Пользователь»;
7. Реализовать функционал загрузки и скачивания файлов, прикрепленных к задачам;
8. Разработать модуль агрегирования данных для вывода аналитической статистики по проектам;
9. Развернуть готовое веб-приложение на удаленном сервере (хостинге).

ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основной целевой аудиторией разрабатываемого веб-приложения являются малые рабочие группы, стартапы, IT-команды на начальном этапе развития и студенческие проектные коллективы. Как правило, численность таких команд составляет от 3 до 20 человек.

В ходе анализа были выявлены основные проблемы (боли) данной аудитории при организации совместной работы традиционными методами:

- Потеря информации и артефактов (файлов) при обсуждении рабочих процессов в неспециализированных мессенджерах;
- Неочевидность статуса выполнения проекта в целом (отсутствие понимания, какой сотрудник занят конкретной задачей в данный момент);
- Высокий порог входа и перегруженность интерфейса в крупных корпоративных системах, что требует излишнего времени на обучение новых сотрудников.

На основе выявленных проблем сформированы ключевые потребности целевой аудитории: потребность в интуитивно понятном интерфейсе, наличии единого пространства для файлов и строгом, но прозрачном разделении зон ответственности.

Для удовлетворения этих потребностей и обеспечения безопасности системы была спроектирована ролевая модель доступа (RBAC). Исходя из паттернов поведения пользователей, выделены две ключевые роли:

- **Администратор (Менеджер проекта):** выступает в роли руководителя или координатора. Обладает полными правами в системе. Занимается созданием новых проектов, управлением списком задач и осуществляет глобальный контроль за ходом выполнения работ через аналитические дашборды.
- **Пользователь (Исполнитель):** выступает в роли рядового участника команды. Имеет ограниченный доступ: видит только те проекты, в которые был приглашен. Основная задача данной роли — брать задачи в работу, изменять их статус по мере выполнения и прикреплять файлы с результатами работы.

2.2 Описание функциональности приложения

Разрабатываемое веб-приложение предоставляет пользователям набор инструментов для совместной работы и эффективного управления проектами. Основная функциональность системы включает в себя:

- Регистрацию, аутентификацию и управление сессиями пользователей;
- Разграничение прав доступа к функциям в зависимости от роли пользователя (администратор или исполнитель);
- Выполнение полного цикла операций (создание, чтение, обновление, удаление) для управления проектами и задачами;
- Загрузку и скачивание файлов, прикрепляемых к конкретным задачам в качестве отчетов;
- Автоматическое агрегирование данных для расчета процента готовности проектов и вывода визуальной статистики.

Взаимодействие пользователей с системой строится вокруг следующего базового сценария: Администратор авторизуется в системе, создает новый проект и формирует список задач, назначая исполнителей. Пользователь, заходя в систему, видит назначенную ему задачу, переводит её в статус «В процессе», а по завершении работы загружает файл с результатом и меняет статус на «Готово». В это же время администратор через дашборд наблюдает за изменением статистики и прогрессом по проекту.

Визуальное представление доступных функций и разграничения прав доступа между ролями продемонстрировано на диаграмме вариантов использования (Use Case) на рисунке 2.1.

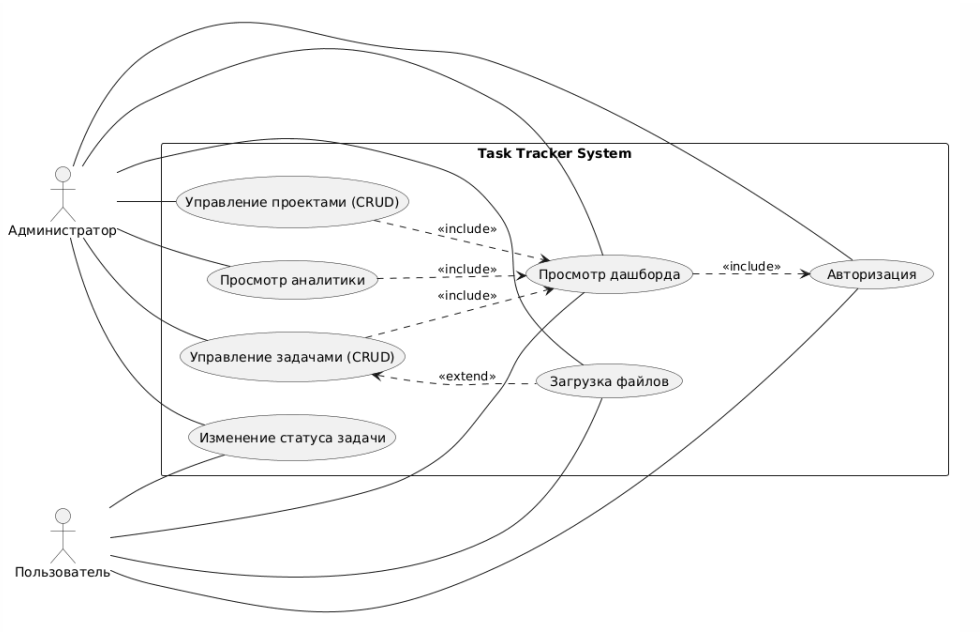

Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

Проектирование базы данных веб-приложения осуществлялось в два этапа: разработка логической и физической схем. В качестве концептуальной основы использовалась ER-модель (модель «Сущность-Связь»). На рисунке 2.2 представлена ER-диаграмма, иллюстрирующая связи между основными объектами системы.

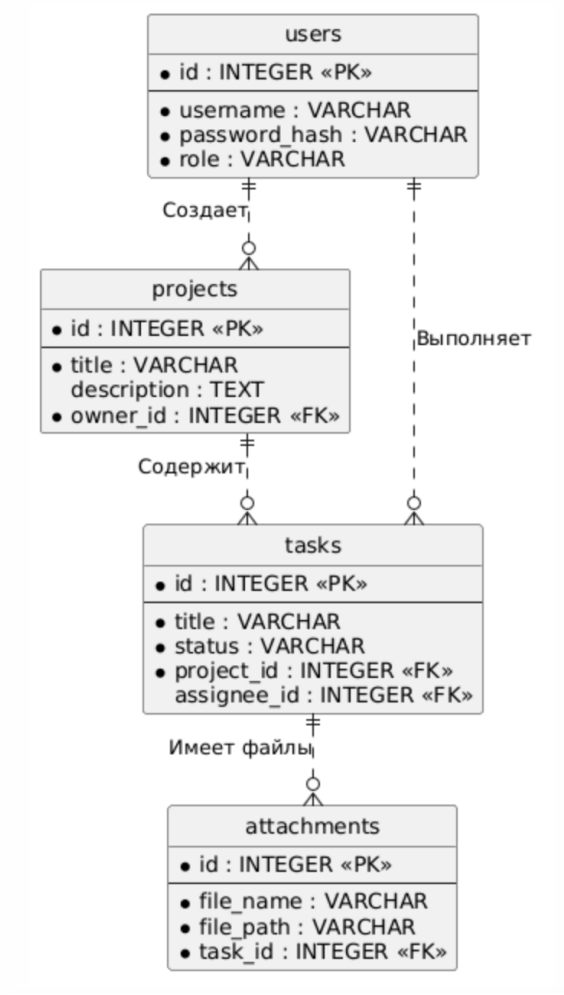

Рисунок 2.2 – ER-диаграмма базы данных

Логическая схема базы данных включает четыре основные сущности:

- `users` (Пользователи) — хранит учетные данные и роль пользователя.
- `projects` (Проекты) — группирует задачи, имеет связь «один-ко-многим» с таблицей пользователей (создатель проекта).
- `tasks` (Задачи) — ключевая сущность, содержащая статус выполнения. Связана с проектами и пользователями-исполнителями.
- `attachments` (Вложения) — сущность, введенная для обеспечения воз-

можности прикрепления множества файлов к одной задаче (связь «один-ко-многим» с таблицей задач).

Физическая схема базы данных описывает реализацию таблиц в СУБД с указанием типов данных и ограничений. Проектирование физической модели выполнено с соблюдением нормальных форм для исключения избыточности данных. Структуры разработанных таблиц представлены ниже.

Таблица 2.1 – Структура таблицы пользователей (users)

Поле	**Тип данных**	**Ограничения и описание**
id	INTEGER	PRIMARY KEY
username	VARCHAR(50)	UNIQUE, NOT NULL
password_hash	VARCHAR(128)	NOT NULL (хэш пароля)
role	VARCHAR(20)	DEFAULT ’user’

Таблица 2.2 – Структура таблицы проектов (projects)

Поле	**Тип данных**	**Ограничения и описание**
id	INTEGER	PRIMARY KEY
title	VARCHAR(100)	NOT NULL
description	TEXT	NULL
owner_id	INTEGER	FOREIGN KEY $\rightarrow$ users.id

Таблица 2.3 – Структура таблицы задач (tasks)

Поле	**Тип данных**	**Ограничения и описание**
id	INTEGER	PRIMARY KEY
title	VARCHAR(100)	NOT NULL
status	VARCHAR(20)	DEFAULT ’pending’
project_id	INTEGER	FOREIGN KEY $\rightarrow$ projects.id
assignee_id	INTEGER	FOREIGN KEY $\rightarrow$ users.id (NULL разрешен)

Таблица 2.4 – Структура таблицы вложений (attachments)

Поле	**Тип данных**	**Ограничения и описание**
id	INTEGER	PRIMARY KEY
file_name	VARCHAR(255)	NOT NULL (Оригинальное имя файла)
file_path	VARCHAR(255)	NOT NULL (Путь на сервере)
task_id	INTEGER	FOREIGN KEY → tasks.id

Связи между таблицами реализованы посредством внешних ключей (Foreign Key). Для обеспечения ссылочной целостности базы данных на уровне ORM SQLAlchemy настроено каскадное удаление (параметр `cascade="all, delete-orphan"`): при удалении проекта или задачи автоматически удаляются все зависимые сущности.

2.4 Разработка макетов страниц

Для обеспечения высокого уровня пользовательского опыта (UX) интерфейс приложения спроектирован в минималистичном стиле. Структура интерфейса позволяет пользователю получить доступ к основным функциям (добавление задачи, просмотр статистики) не более чем за два клика.

На рисунке 2.3 представлен макет панели управления (дашборда), а на рисунке 2.4 — макет детальной страницы проекта с таблицей задач.

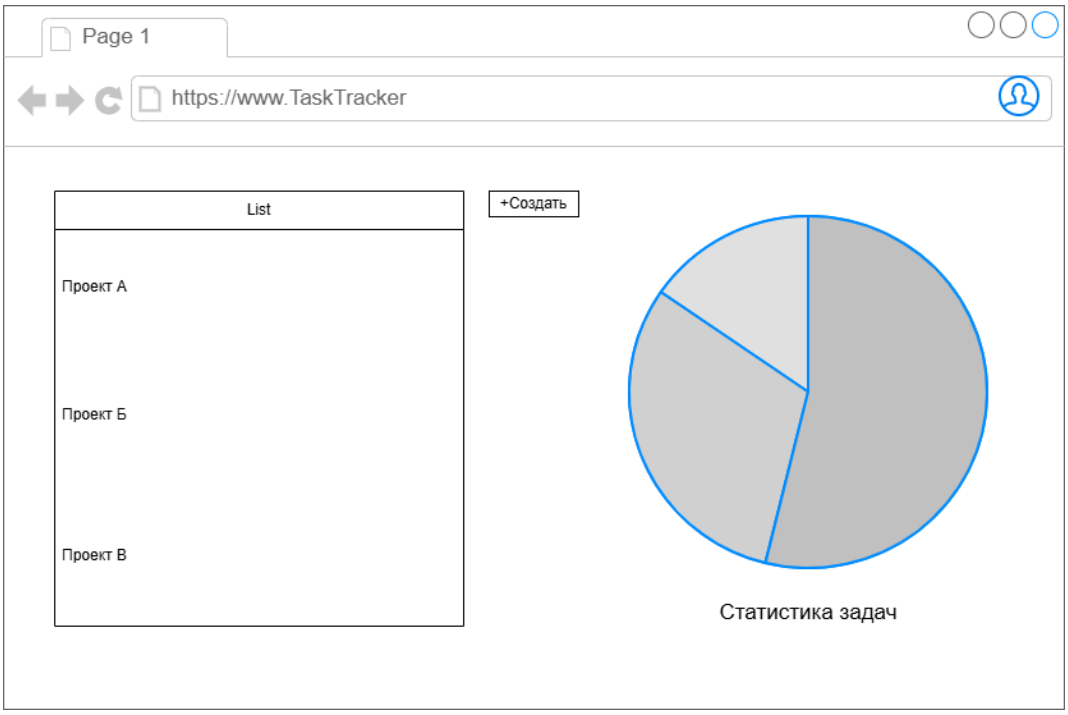

Рисунок 2.3 – Макет главной панели управления (Дашборд)

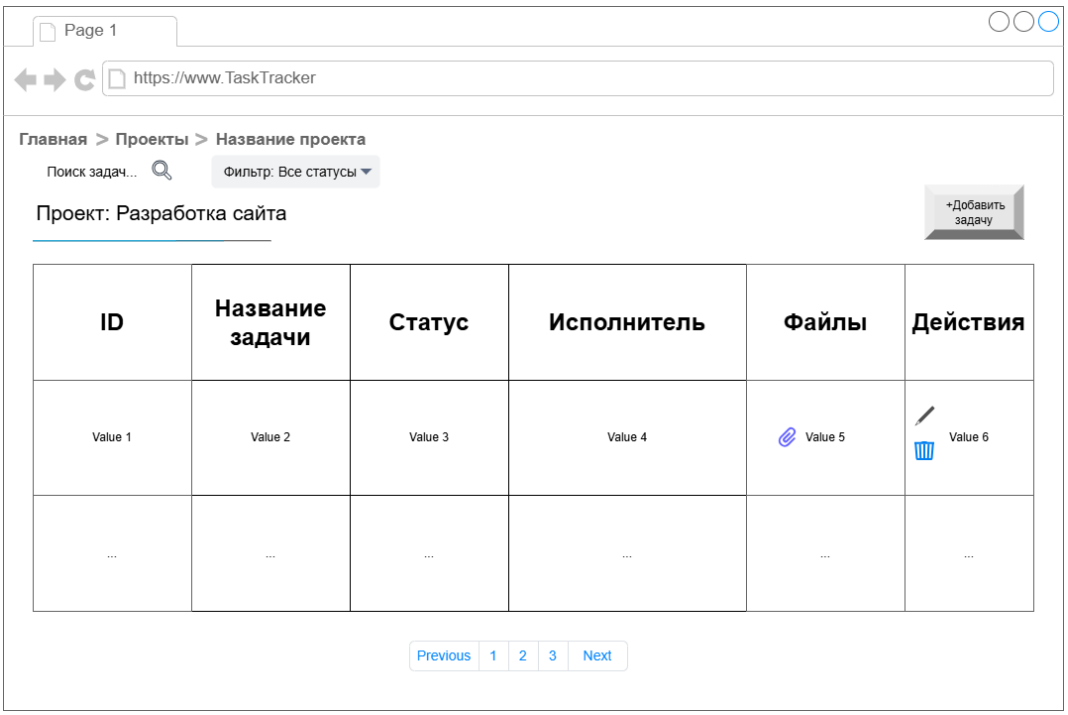

Рисунок 2.4 – Макет страницы проекта со списком задач

РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Реализация подсистемы аутентификации и проверки ролей

Основой безопасности разрабатываемого приложения является подсистема аутентификации, реализованная с помощью библиотеки `Flask-Login`. Хранение паролей в базе данных осуществляется в зашифрованном виде с использованием криптографических хэш-функций из модуля `werkzeug.security` (функции `generate_password_hash` и `check_password_hash`).

Для разграничения прав доступа была внедрена ролевая модель (RBAC). При выполнении критически важных действий (например, создание или удаление проекта) на стороне сервера происходит проверка атрибута `role` текущего пользователя.

Листинг 1 — Фрагмент кода проверки прав доступа при создании проекта

```
@app.route('/create_project', methods=['POST'])
@login_required
def create_project():
    if current_user.role != 'admin':
        flash('У вас нет прав для создания проектов.', 'danger')
        return redirect(url_for('index'))

    title = request.form.get('title')
    description = request.form.get('description')

    if title:
        new_project = Project(title=title, description=description,
                              owner_id=current_user.id)
        db.session.add(new_project)
        db.session.commit()
    return redirect(url_for('index'))
```

На рисунке 3.1 представлен интерфейс страницы входа в систему.

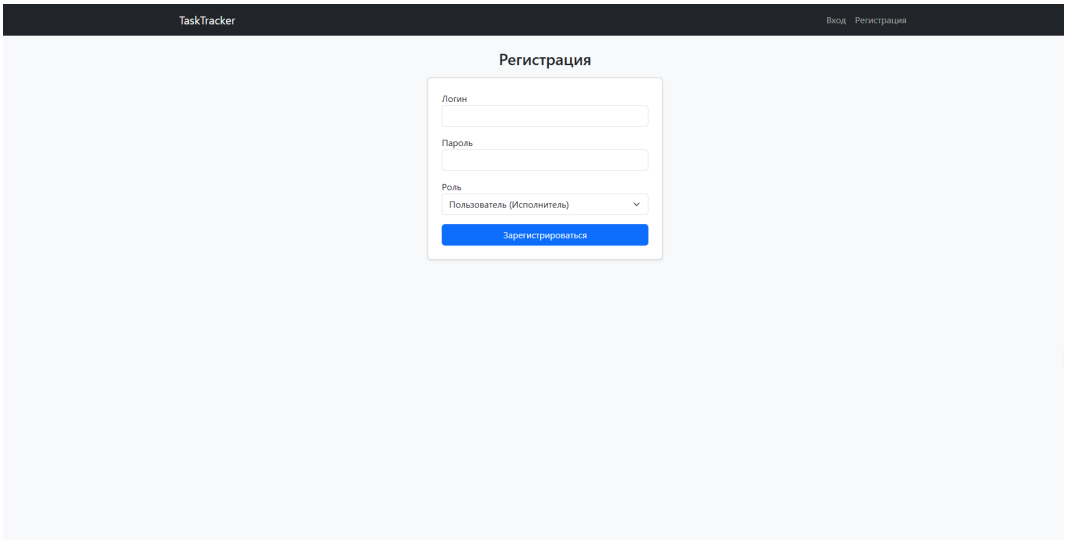

Рисунок 3.1 – Интерфейс страницы регистрации пользователей

3.2 Разработка CRUD-интерфейсов и работы с файлами

Управление сущностями системы осуществляется через реализованные CRUD-интерфейсы (создание, чтение, обновление, удаление). Взаимодействие с реляционной базой данных PostgreSQL инкапсулировано через объектно-реляционное отображение (ORM) `SQLAlchemy`. Настроено каскадное удаление данных: при удалении проекта из БД автоматически удаляются все связанные с ним задачи.

Особое внимание уделено безопасной работе с файлами и возможности **множественного прикрепления документов** к одной задаче. Информация о загруженных файлах сохраняется в отдельную связанную таблицу. Для защиты файловой системы сервера используется утилита `secure_filename`, а для предотвращения коллизий (перезаписи файлов с одинаковыми именами) реализована генерация уникальных префиксов на базе библиотеки `uuid`. Кроме того, внедрен механизм автоматического физического удаления файлов с жесткого диска при удалении проекта или задачи, что предотвращает утечки памяти на сервере.

На рисунке 3.2 продемонстрирован интерфейс управления задачами внутри проекта, включающий возможности фильтрации, пагинации и управления жизненным циклом задачи.

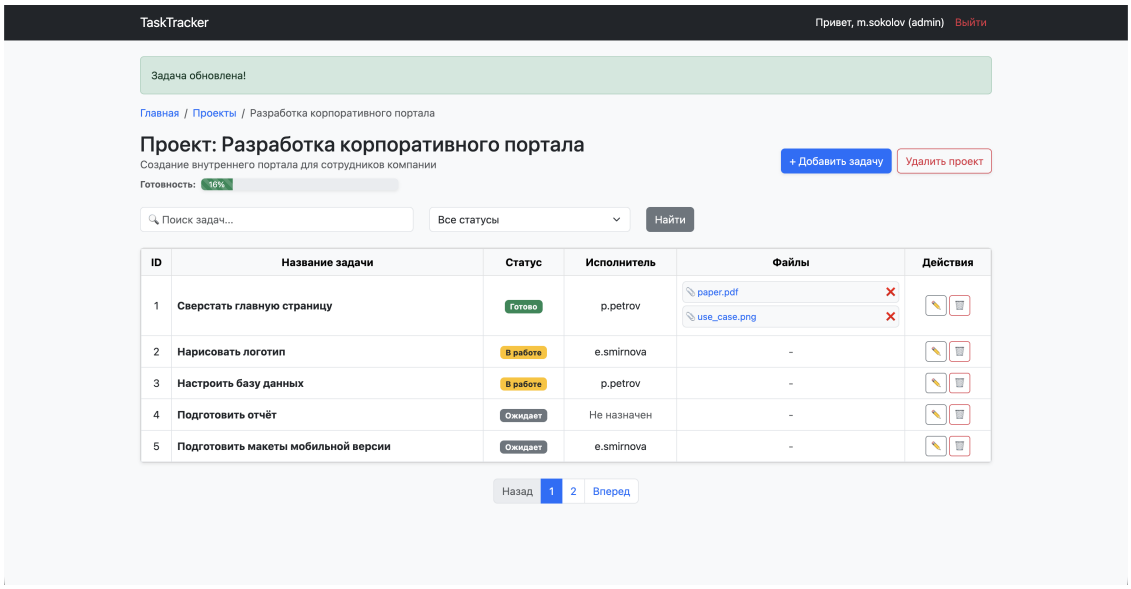

Рисунок 3.2 – Интерфейс управления задачами проекта

3.3 Реализация модуля агрегирования данных

Для обеспечения администратора системы наглядной аналитикой был разработан модуль агрегирования данных. На серверной стороне с помощью агрегатных функций SQLAlchemy (метод `.count()`) выполняется подсчет задач, сгруппированных по их текущему статусу.

Полученные статистические данные передаются в HTML-шаблоны, где визуализируются в двух форматах:

- В виде динамической шкалы готовности (Progress Bar) внутри каждого проекта, отражающей процент выполненных задач.
- В виде интерактивной круговой диаграммы на базе библиотеки `Chart.js` на главной панели управления.

Листинг 2 — Фрагмент кода агрегации данных в базе

```
# Агрегирование данных для круговой диаграммы
total_tasks = Task.query.count()
pending_tasks = Task.query.filter_by(status='pending').count()
in_progress_tasks = Task.query.filter_by(status='in_progress').count()
done_tasks = Task.query.filter_by(status='done').count()

stats = {
    'total': total_tasks,
    'pending': pending_tasks,
    'in_progress': in_progress_tasks,
    'done': done_tasks
}
```

Результат работы модуля визуализации и агрегации представлен на главной панели управления системой (рисунок 3.3).

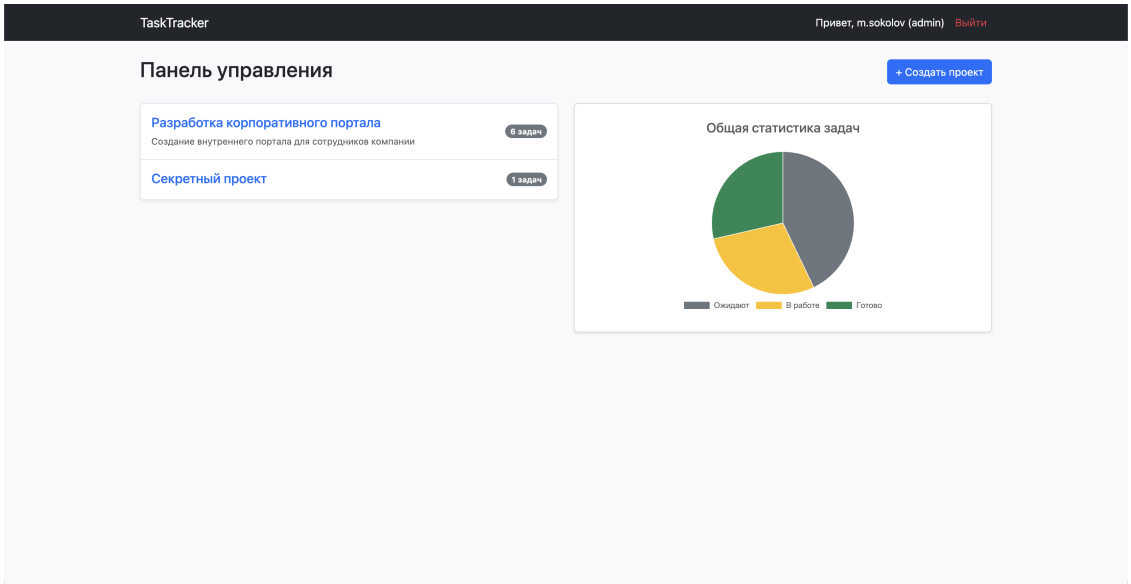

Рисунок 3.3 – Главная панель управления (Дашборд) с круговой диаграммой

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно достигнута поставленная цель — спроектировано и разработано полноценное клиент-серверное веб-приложение для управления проектами и отслеживания задач.

В результате выполнения работы были решены следующие задачи:

1. Проведен анализ предметной области, который подтвердил актуальность создания легковесных и интуитивно понятных систем управления задачами (Task Trackers) для малых рабочих групп.

2. Спроектирована архитектура приложения с использованием микрофреймворка Flask (Python) и объектно-реляционного отображения SQLAlchemy, обеспечивающая высокую производительность и безопасность взаимодействия с базой данных PostgreSQL.

3. Разработаны пользовательские интерфейсы (Wireframes), которые впоследствии были успешно перенесены в интерактивные HTML-шаблоны с применением CSS-фреймворка Bootstrap 5.

4. Внедрена надежная подсистема аутентификации и реализована ролевая модель (RBAC), разделяющая функционал системы между «Администраторами» и обычными «Пользователями».

5. Реализован полный цикл CRUD-операций для трех ключевых сущностей предметной области: Пользователи, Проекты и Задачи.

6. Интегрирован функционал безопасной загрузки, хранения и скачивания файлов-отчетов, прикрепляемых к конкретным задачам.

7. Разработан аналитический модуль агрегирования данных, который в реальном времени визуализирует статистику выполнения задач с использованием графической библиотеки Chart.js.

Разработанная система полностью удовлетворяет техническим требованиям, отличается высокой скоростью отклика, защищенностью (шифрование паролей, защита от SQL-инъекций) и адаптивным дизайном, и может служить надежным инструментом для автоматизации рабочих процессов в малых и средних командах.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [3], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

[1] Flask Documentation (3.0.x) [Электронный ресурс] [Text]. — URL: https://flask.palletsprojects.com/ (дата обращения: 01.04.2026).

[2] SQLAlchemy 2.0 Documentation[Электронный ресурс] [Text]. — URL: https://docs.sqlalchemy.org/ (дата обращения: 01.04.2026).

[3] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.